AgriSort

Nathan Puskuri, Bimlendra Ray, Praharsha K

***Abstract*— This paper presents the design and implementation of an automated robotic system utilizing a fetch manipulator equipped with computer vision abilities for sorting fruits and vegetables on a coffee table. As the produce is spawned on the table, the system classifies each item based on visual characteristics. It employs a dual-bin architecture, where fruits are placed in the left bin and vegetables in the right bin thereby facilitating quick retrieval. This approach integrates robotic movement, and computer vision, aiming to enhance automation in agricultural processing and improve operational efficiency. It contributes to the field of robotic automation, offering insights into the development of intelligent systems for agricultural applications.**

I. INTRODUCTION

The agriculture and food processing industries are undergoing a significant transformation driven by technological advancements and the increasing demand for efficiency and sustainability.The need for innovative solutions to enhance food production and distribution is more critical than ever. Traditional methods of sorting fruits and vegetables, which rely heavily on manual labor, are increasingly being challenged by the need for speed, accuracy, and cost-effectiveness. As such, there is a pressing demand for automated systems that can streamline these processes.

Manual sorting not only consumes valuable time but also introduces a high degree of variability and potential errors. Workers may misclassify items based on fatigue or oversight, leading to reduced quality control and customer dissatisfaction.Labor shortages in many regions add to these challenges, making it difficult for producers to maintain efficient operations. By automating the sorting process, we can significantly reduce these issues, ensuring that only the highest-quality produce reaches consumers while simultaneously reducing operational costs.

The integration of robotics and artificial intelligence in agricultural practices presents a transformative opportunity to address these challenges. Automated systems can operate continuously, significantly increasing throughput while maintaining a consistent level of accuracy. By employing image recognition and machine learning algorithms, we can develop a system capable of identifying and sorting various fruits and vegetables based on visual characteristics, thereby enhancing the efficiency of sorting operations. Moreover, this project aligns with the broader movement toward sustainable agriculture. By improving sorting efficiency, we can reduce food waste,a significant global issue,by ensuring that produce is classified correctly and handled appropriately. Proper sorting also facilitates better supply chain management, allowing for more efficient distribution and reducing the environmental impact associated with food transportation and spoilage.

Additionally, the use of robotic systems in agriculture opens avenues for research and development in related fields, such as computer vision, machine learning, and human-robot interaction. As we explore the potential of these technologies, we can contribute valuable insights that can be applied to various domains beyond agriculture, fostering innovation in automation and robotics.

In summary, the motivation for this project stems from the need to modernize sorting processes in the agricultural sector. By developing an automated robotic system that can efficiently and accurately sort fruits and vegetables, we aim to enhance productivity, reduce labor costs, and minimize food waste, ultimately contributing to a more sustainable and efficient food supply chain. This endeavor not only addresses pressing industry challenges but also represents an exciting opportunity to advance the field of robotics and machine learning, paving the way for future innovations in agricultural automation.

A. Overview of the Framework:

This project proposes an automated system with a Fetch manipulator equipped with a camera and a conveyor belt for sorting mixed fruits and vegetables. The system consists of several key components:

1) **Table:** A variety of fruits and vegetables are spawned on the coffee table , closer to the robot, ensuring the items are in reach of robot for sorting.
2) **Camera System:** The robot is equipped with a high-resolution camera that captures images of the items on the conveyor belt in real-time. This enables the system to gather visual data necessary for classification.
3) **Image Processing:** Captured images are processed using AWS Rekognition, an in-house Image classification system completely managed by Amazon. The images undergo pre-processing steps such as resizing, normalization, and filtering to enhance their quality for analysis.
4) **Object Classification:** The processed images are analyzed using the AWS Rekognition to identify all possible objects in the image.All identified objects are associated with a label and confidence and fruit/vegetable classification is employed through code based on labels identified.
5) **Robotic Manipulation:** Upon identification, the Fetch manipulator is programmed to accurately pick up the sorted items and deposit them into designated bins—fruits on the left and vegetables on the right of the robot.

II. RELATED WORK

The case of automation of fruit harvesting and sorting has been studied in where 2 robotic arms are employed to perform distinct functions[1].The first robotic arm,the one closest to the simulated fruit tree is responsible for picking the fruits from the tree and place them on the conveyor belt which then moves it towards the second robot arm.The second arm is responsible to pick the fruit from conveyor belt and determine the quality of the fruit.If the fruit is determined to be of good quality,it is placed in fruit collection area and in a specific area.Both the arms communicate through a socket over TCP/IP protocol.Depth cameras are employed on both the robotic arms - the first arm uses depth camera to identify the location of the fruit and the second arm uses depth camera to understand the quality of fruit- whether good or bad. The system uses techniques like edge detection,color analysis and shape recognition to identify fruits and their corresponding coordinates.The quality of fruits is assessed by checking color uniformity,defects and size measurements.After these checks are made and the quality of fruit is assessed,the second arm grasps the object based on coordinates and places it in corresponding bins.

The applications of computer vision techniques in fruit picking robots discusses the research results of computer vision in fruit picking robots in aspects of imaging equipment, processing algorithms and localization of target fruits [2]. Different imaging equipments like monocular camera, binocular stereo camera, spectral camera and their image qualities are discussed. Once the image is captured, fruit image processing algorithms are employed to understand fruit localization.Various features like color difference between fruits and background, color similarity between fruit and background ,Uneven illumination and Occlusion are explained as these features affect the detection algorithm used.Once the image has been processed,target fruit localization techniques are leveraged to understand exact location of fruit in the scene. However,we do not plan to localize the fruits unlike the elephant robotics experiment as we have predefined location for the fruits on the conveyor belt in our system. We plan to utilize OpenCV algorithm that takes in image of fruit/vegetable - processes it and determines if it is a fruit or vegetable. Once that decision has been made,we utilize the robot gripper to grasp the fruit and place it in corresponding bin.

Visual feature analysis and classifiers for fruit classification emphasizes the analysis of visual cues to automate the visual inspection and packaging processes [3].Different color,shape and texture features are analyzed by comparing various supervised machine learning algorithms.The dataset is divided into training and test dataset and the train dataset is used to perform image pre-processing techniques like smoothing to reduce noise and illumination.Various features like color,texture,shape are extracted and sent to image classification algorithm for training.Once trained,it is tested on test dataset for performance evaluation. The Lightweight and efficient deep learning models for fruit detection in orchards article evaluates lightweight models for fruit detection in orchards using YOLOv5 algorithm that is a single-stage network designed for one-shot detection[4].We plan to follow similar architecture through OpenCV for image preprocessing and YOLOv5(You Only Look Once) algorithm for image classification.

III. METHOD

To solve the problem of automated sorting, we have planned four key features:

1) MoveIt! for object grasping and manipulation
2) Capture image of environment using fetch robot's camera
3) Classifying each object as a fruit or vegetable with AWS Rekognition
4) Estimating the 6D pose of objects for accurate positioning and orientation

A. Feature 1: MoveIt! for object grasping and manipulation

In this work, we utilize the **MoveIt** motion planning framework to control a robotic arm for object manipulation tasks, specifically moving objects into a designated bin. The process involves precise planning and execution of trajectories to ensure safe and efficient object handling. MoveIt provides tools to define target poses for the robot's end-effector, plan collision-free paths, and execute those paths with high reliability. In addition to defining target poses, we also use MoveIt's Cartesian path planning capabilities to create smooth and precise motions when moving the gripper towards the object.

First, we get the pose of the object that needs to be grasped. The object is then rotated along the Y-axis. This is done because when we compute the grippers' target pose, we use the objects pose. By rotation along the Y-axis the gripper target pose is also rotated. This means that our gripper will grasp the object from above now. This is important because often times the robot arm will collide with the table without this step being done.

Next, we use **offsets** to maintain consistent and collision-free operations. An array of offsets are created based on the objects rotated pose with the first offset being the greatest and the last offset being the smallest. Without these adjustments, the gripper sometimes hits the table during grasping or plans a path that ends up colliding with the table. The offsets allow the gripper to move further above the objects, leading a consistent path that avoids the table.

After defining the array of offsets, we use MoveIts various methods such as setposetarget and plan in order to move to the first offset. However, we use cartesian path planning on the rest of the offsets to create a list of waypoints. These waypoints are then used by MoveIt in order to move to the goal pose. Using cartesian paths appeared preferable to setposetarget and plan in this case. The cartesian paths allow the robot's end effector follows a series of waypoints ensuring a smooth linear motion that avoids abrupt deviations and reduces the risk of collisions. This is particularly critical, because sometimes when using setposetarget and

plan MoveIt can choose various non-direct trajectories that can result in collisions with the table. Using the cartesian paths provides a simple, consistent, and direct approach to the target object.

By defining consistent approach distances with offsets and using cartesian paths to compute waypoints for MoveIt, the system avoids potential errors, such as misalignment or unintentional contact with surfaces.

In the midterm report it was mentioned that GraspIt would be used to compute the best grasps. These grasps were to be used with MoveIt to plan the movement of the object. However, there were complications regarding dependencies when attempting to implement GraspIt and implementation seemed unlikely. The approach to this project was changed, and instead the best grasps for each object was found by rotating the objects poses by Y-axis, and then we would manually tests offsets for each object in order to find the most consistent offsets.

B. Feature 2: Capturing Image using fetch robot's camera

We are using the camera attached to fetch robot's head to capture images of the environment and thereby identify if it has a fruit/vegetable placed on the table in front of it.These raw rgb images are written to a topic /head-camera/rgb/image-raw and a subscriber subscribes to this topic to receive images in real-time.The subscriber keeps running to receive images continuously.

C. Feature 3: Object Classification with AWS Rekognition

Once the subscriber node receives an image from the topic it has subscribed to,it calls AWS Rekognition's detect-labels method to return all identified object labels in the image along with corresponding confidence score.Once the labels are received from Rekognition,they are processed to understand if the image contains any fruit/vegetable.

D. Feature 4: 6D Pose Estimation

This feature is a stretch goal. Instead of assuming known positions and orientations, we aim to use a machine learning algorithm capable of estimating the 6D pose of objects. This would likely involve using a camera to capture an image and applying pose estimation algorithms to determine the object's precise location and orientation. One way we can implement this is by using the graspnetAPI. This API which is given a 3D point cloud of an object will return a 6D pose estimate for an object[5][6]. Due to time constraints this stretch goal was not implemented.

IV. Experiments

Our project will be built using ROS (Robot Operating System) with RViz for 3D robot visualization and Gazebo for simulation. Gazebo provides a complete development toolbox that improves both efficiency and accuracy in our simulations.

In this simulation, a Fetch robot with interchangeable end-effectors is positioned in front of a coffee table , with produce being spawned on the table, with items placed randomly.

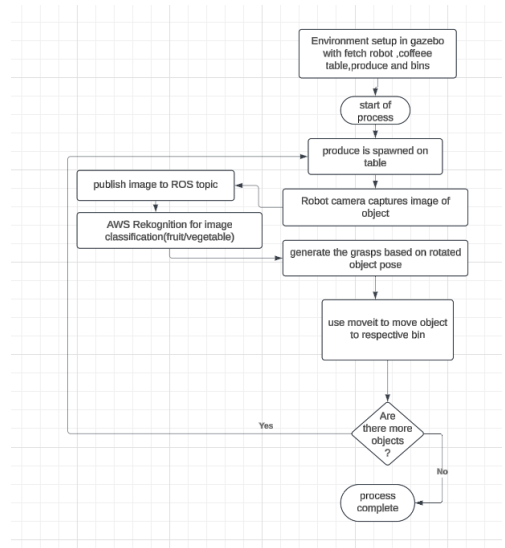

Fig. 1. Flow-chart of process execution

Containers for fruits and vegetables are located on the robot's right and left sides, respectively. The robot identifies, picks and sorts each item, placing it in the corresponding bin based on type. A Python script is used to populate the table with fruits and vegetables randomly, with each object having random sizes as well.

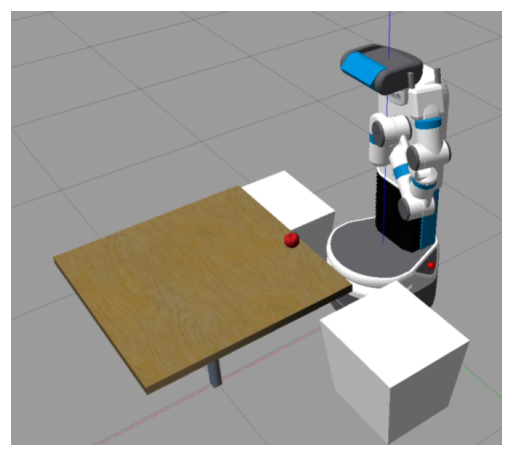

Fig. 2. Experimental setup with a fruit(apple)

Currently, in the initial setup, we are starting with a table, since our focus is to get GraspIt! and MoveIt! Working first. Our goal here is to sort fruits and vegetables using an image classification algorithm that identifies objects on the moving conveyor belt with the help of a robot arm equipped with an end-effector to move the objects into the appropriate bin.

We will be testing four different end-effectors: a fast, robust, and user-friendly parallel jaw gripper, and three more complex, resource-intensive options—the Barrett hand, DLR hand, and Robonaut hand—which, though versatile, operate more slowly than the parallel gripper. This project will evaluate each gripper to determine the optimal choice for sorting fruits and vegetables. Finally, we will be using the

selected gripper for our future work.

We plan to evaluate the robot's performance through various metrics like object detection accuracy, object classification accuracy, throughput, and sorting accuracy. We aim to achieve high object detection accuracy, high object classification accuracy, high sorting accuracy, and high throughput.

A. Robustness

Robustness measures the ability of a robot to correctly identify the objects when the color of the conveyor belt is changed. The robustness is defined as:

$$\text{Robustness}_{\text{color}} = \frac{\text{Number of objects correctly detected}}{\text{Total number of objects}} \times 100$$

B. Object Classification Accuracy

The accuracy of the OpenCV algorithm to correctly identify an object as either a fruit or a vegetable is calculated as:

$$\text{Accuracy}_{\text{classification}} = \frac{\text{Number of correctly classified objects}}{\text{Total number of objects}} \times 100$$

C. Sorting Accuracy

The percentage of correctly classified and sorted objects placed in the correct bin is defined as:

$$\text{Accuracy}_{\text{sorting}} = \frac{\text{Correctly sorted objects}}{\text{Total number of objects sorted}} \times 100$$

D. Throughput

Throughput, defined as the number of objects sorted in a unit amount of time, is given by:

$$\text{Throughput} = \frac{\text{Total number of objects sorted}}{\text{Total time taken for completion of task}}$$

E. Results

$$\text{Robustness}_{\text{color}} = 100 percent$$

$$\text{Accuracy}_{\text{classification}} = 100 percent$$

$$\text{Accuracy}_{\text{sorting}} = 100 percent$$

$$\text{Throughput} = \frac{2}{120} seconds = \frac{1}{60} seconds$$

V. Conclusion

The AgriSort system demonstrates a practical approach to automating the sorting of fruits and vegetables using robotic manipulation and computer vision. By integrating MoveIt! for motion planning and AWS Rekognition for classification, the project achieves reliable sorting accuracy and efficiency. While some planned features, like 6D pose estimation, were not implemented, the project met its primary objectives and provides a strong base for further development. Lessons learned from challenges, such as collision handling and dependency management, will guide improvements in future iterations.

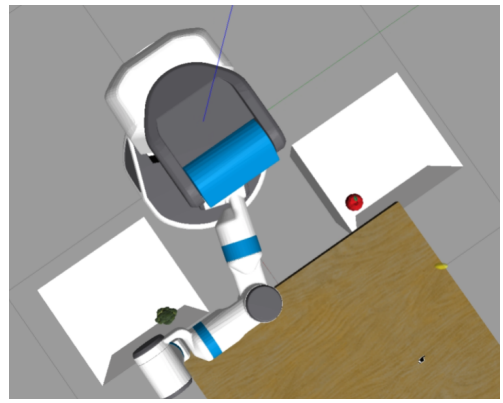

Fig. 3. Result of sorting process

A. Take Home Message

This project shows how robotics can address real-world challenges in agriculture by reducing manual labor and improving sorting efficiency. The methods applied here, though simple, highlight the potential of combining robust robotic control with reliable object classification systems. There is significant room for growth, but even at this stage, AgriSort offers valuable insights into the use of robotics in agricultural automation.

B. Running the Programs

In order to run the code we used the same steps in the homework. Run the python file fetch image2 after starting the gazebo environment and running MoveIt.

References

[1] "Automating fruit harvesting and sorting: Educational case." [Online]. Available: https://shop.elephantrobotics.com/blogs/news/automating-fruit-harvesting-and-sorting-educational-case

[2] H. Si, J. Lv, K. Lin, J. Wu, and J. Chen, "A review of application of computer vision in fruit picking robot," in *Advances in Intelligent Systems and Computing*. Springer, 2020, pp. 1–7.

[3] Y. Zhang, J. Wang, and H. Li, "Analysis of visual features and classifiers for fruit classification problem," *Computers and Electronics in Agriculture*, vol. 182, p. 106267, 2021.

[4] Z. Wang, Y. Yang, and M. Liu, "Lightweight and efficient deep learning models for fruit detection in orchards," *Scientific Reports*, vol. 14, p. 76662, 2024.

[5] H.-S. Fang, C. Wang, M. Gou, and C. Lu, "Graspnet-1billion: A large-scale benchmark for general object grasping," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition(CVPR)*, 2020, pp. 11 444–11 453.

[6] H.-S. Fang, M. Gou, C. Wang, and C. Lu, "Robust grasping across diverse sensor qualities: The graspnet-1billion dataset," *The International Journal of Robotics Research*, 2023.